

Vue3 全栈前端高频八股文档

合并说明：本文件由 codex-frontend-interview-vue3.md（主体）+ claude-mimo-frontend-interview-guide.md 合并整理，后者的独特内容见第九节补充。

适用定位：5 年 C# 后端开发，转苏州全栈岗位，前端方向以 Vue3 + Pinia + Vue Router + Axios + Element Plus 后台系统为主。

回答原则：偏面试话术，先说结论，再说项目落地，最后补一句风险或边界。

一、Vue3 核心

1. Vue3 响应式原理是什么？

问题：Vue3 为什么数据变化后页面会自动更新？

标准答案：Vue3 基于 Proxy 做响应式代理。组件渲染时读取数据会进行依赖收集，数据修改时会触发依赖更新，Vue 再异步批量更新 DOM。相比 Vue2 的 Object.defineProperty，Vue3 对对象新增属性、数组、Map、Set 的支持更自然。

面试加分点：可以提到依赖收集是 track，触发更新是 trigger；模板渲染本质上也是一个响应式副作用。

常见误区：认为 Vue 是定时轮询或脏检查；认为响应式数据改完后 DOM 会立刻同步更新。

代码示例：

```
import { reactive, effect } from 'vue'

const state = reactive({ count: 0 })

effect(() => {
  console.log(state.count)
})

state.count++
```

2. ref 和 reactive 有什么区别？

问题：什么时候用 ref，什么时候用 reactive？

标准答案：ref 适合基本类型，也可以包装对象，脚本中访问需要 .value；reactive 适合对象、数组这类复杂结构，返回代理对象。实际后台系统中，loading、visible 常用 ref，查询条件、表单对象常用 reactive。

面试加分点：reactive 解构会丢响应式，需要用 toRefs，或者保持 query.xxx 这种访问方式。

常见误区：把 reactive 对象直接解构后还以为字段保持响应式。

代码示例：

```
const loading = ref(false)

const query = reactive({
```

```
    pageIndex: 1,
    pageSize: 10,
    keyword: ''
  })
```

3. computed 和 watch 的区别?

问题：计算属性和监听器怎么选？

标准答案：computed 用来根据已有状态派生新值，有缓存，依赖不变不会重新计算；watch 用来监听变化后执行副作用，比如请求接口、写缓存、联动表单。能描述成“一个值”的用 computed，需要“做一件事”的用 watch。

面试加分点：watch 默认不会立即执行，需要 immediate: true；深监听 deep 要谨慎，容易带来性能问题。

常见误区：用 watch 维护一个本可以用 computed 得出的字段，导致状态冗余。

代码示例：

```
const fullName = computed(() => `${user.firstName}${user.lastName}`)

watch(
  () => query.keyword,
  () => {
    query.pageIndex = 1
    loadData()
  }
)
```

4. watch 和 watchEffect 的区别?

问题：watchEffect 适合什么场景？

标准答案：watch 需要明确监听源，适合业务代码里精确控制；watchEffect 会自动收集回调中用到的响应式依赖，并且创建后立即执行。后台项目里接口请求通常优先用 watch，依赖更清楚。

面试加分点：异步回调里，watchEffect 只会收集首次 await 之前访问到的依赖。

常见误区：复杂接口请求滥用 watchEffect，导致请求触发条件不清楚。

代码示例：

```
watchEffect(() => {
  console.log(query.pageIndex, query.keyword)
})
```

5. nextTick 的作用?

问题：为什么修改数据后马上拿 DOM，拿到的还是旧状态？

标准答案：Vue 的 DOM 更新是异步批量执行的。数据变化后，Vue 会把更新放进队列，等当前同步代码执行完再刷新 DOM。nextTick 用来等待本轮 DOM 更新完成，常用于弹窗打开后聚焦、表格滚动、获取元素尺寸。

面试加分点：Vue 这样做是为了合并多次状态变更，减少重复渲染。

常见误区：把 `nextTick` 当成普通延时器，它等的是 Vue 更新队列，不是固定时间。

代码示例：

```
const visible = ref(false)
const inputRef = ref<HTMLInputElement>()

async function open() {
  visible.value = true
  await nextTick()
  inputRef.value?.focus()
}
```

6. Vue3 组件通信方式有哪些？

问题：父子、兄弟、跨层级组件怎么传值？

标准答案：父传子用 `props`，子传父用 `emit`；跨层级少量数据可以用 `provide/inject`；跨页面或全局共享状态用 `Pinia`。兄弟组件通常把状态提升到共同父组件，或者使用 `store`。

面试加分点：`<script setup>` 中常用 `defineProps`、`defineEmits`、`defineExpose`，新项目也可能用 `defineModel` 简化 `v-model`。

常见误区：子组件直接修改 `props`；把事件总线当成全局状态管理，后期难排查。

代码示例：

```
<script setup lang="ts">
defineProps<{ title: string }>()
const emit = defineEmits<{ save: [id: number] }>()
</script>

<template>
  <el-button @click="emit('save', 1)">保存</el-button>
</template>
```

7. v-if 和 v-show 的区别？

问题：权限按钮、搜索区域、弹窗内容该用哪个？

标准答案：`v-if` 是条件渲染，不满足时组件不会创建；`v-show` 是 CSS 显隐，组件一直存在。切换频繁用 `v-show`，权限控制或不常显示的内容用 `v-if`。

面试加分点：权限按钮建议用 `v-if`，但前端只控制显示，真正权限必须后端校验。

常见误区：认为 `v-show` 隐藏后就安全了，实际上 DOM 仍然存在。

代码示例：

```
<el-button v-if="hasPermission('user:add')">新增</el-button>
<el-form v-show="showSearch" />
```

8. v-for 为什么要写 key？

问题：为什么不建议动态列表用 `index` 当 `key`？

标准答案：key 用来标识节点身份，帮助 Vue 在 diff 时正确复用和移动节点。动态列表如果用 index，新增、删除、排序时容易导致组件状态、输入框内容、选中状态错位。

面试加分点：后台表格行编辑、可勾选列表、动态表单项都应该使用稳定业务 ID。

常见误区：认为 key 只是为了消除控制台 warning。

代码示例：

```
<div v-for="item in users" :key="item.id">
  {{ item.name }}
</div>
```

9. Vue3 常见性能优化有哪些？

问题：后台页面卡顿怎么排查和优化？

标准答案：先用 Network、Performance、Vue Devtools 定位问题，再优化。常见手段包括分页加载、接口防抖、路由懒加载、组件拆分、减少深监听、大列表虚拟滚动、避免一次渲染过多 DOM。

面试加分点：后台系统很多卡顿不是 Vue 本身问题，而是表格数据太多、接口重复请求、状态设计不合理。

常见误区：没有定位瓶颈就直接上复杂优化，甚至增加维护成本。

代码示例：

```
const UserPage = () => import('@/views/user/index.vue')
```

二、Vue Router 与 RBAC 权限

10. hash 模式和 history 模式区别？

问题：Vue Router 两种模式怎么选？

标准答案：hash 模式 URL 带 #，刷新不会请求后端对应路径，部署简单；history 模式 URL 更美观，但刷新页面需要服务器配置 fallback 到 index.html。后台系统常用 history，但部署时要配 Nginx 或网关重写。

面试加分点：能说出生产环境刷新 404 通常是 history 模式没有配置 fallback。

常见误区：本地正常、线上刷新 404，就以为是前端路由写错了。

代码示例：

```
location / {
  try_files $uri $uri/ /index.html;
}
```

11. 路由守卫如何做登录校验？

问题：未登录访问后台页面如何拦截？

标准答案：在 beforeEach 中判断 token。白名单页面直接放行；无 token 跳登录；有 token 但用户信息或权限未加载时，先拉取用户信息和动态路由，再进入目标页面。

面试加分点：Vue Router 4 中可以直接 return 路径或布尔值，避免 next 多次调用。

常见误区：动态路由未注册就放行，刷新后直接 404。

代码示例：

```
router.beforeEach(async (to) => {
  if (whitelist.includes(to.path)) return true
  if (!userStore.token) return `/login?redirect=${to.fullPath}`

  if (!userStore.loaded) {
    await userStore.loadUserAndRoutes()
    return to.fullPath
  }

  return true
})
```

12. 基于路由的 RBAC 怎么设计？

问题：后台系统菜单、路由、按钮权限怎么做？

标准答案：后端根据用户角色返回菜单树、路由标识和按钮权限码。前端把菜单转换为动态路由，通过 `addRoute` 注册；侧边栏根据菜单渲染；按钮用权限码函数或指令控制显示。接口权限和数据权限必须由后端兜底。

面试加分点：能区分菜单权限、路由权限、按钮权限、接口权限、数据权限。

常见误区：只隐藏按钮但后端不鉴权，用户仍可以直接调用接口。

代码示例：

```
function hasPermission(code: string) {
  return userStore.permissions.includes(code)
}
```

```
<el-button v-if="hasPermission('order:refund')">退款</el-button>
```

13. 动态路由刷新丢失怎么办？

问题：为什么刷新后动态菜单页面访问 404？

标准答案：动态路由是运行时添加的，刷新后内存丢失。需要在应用初始化或路由守卫中，根据 token 重新请求用户菜单和权限，重新注册动态路由，注册完成后再进入目标地址。

面试加分点：退出登录时要清理 token、store 和动态路由，避免切换账号权限污染。

常见误区：把路由存在 localStorage 就认为安全，实际权限仍应以后端最新返回为准。

代码示例：

```
const routes = transformMenusToRoutes(menus)
routes.forEach(route => router.addRoute('Layout', route))
```

三、Pinia 状态管理

14. 哪些状态适合放 Pinia?

问题：Pinia 和组件本地状态怎么取舍？

标准答案：跨页面共享、刷新后需要恢复、多个模块依赖的状态适合放 Pinia，比如 token、用户信息、权限码、字典数据。单页面的表单、弹窗、分页参数一般放组件内部。

面试加分点：store 不是数据垃圾桶，过度全局化会让状态来源不清晰。

常见误区：所有接口返回都放 Pinia，导致缓存脏数据和页面互相影响。

代码示例：

```
export const useUserStore = defineStore('user', () => {
  const token = ref('')
  const permissions = ref<string[]>([])

  function logout() {
    token.value = ''
    permissions.value = []
  }

  return { token, permissions, logout }
})
```

15. Pinia 持久化怎么做？

问题：刷新后 token 丢失怎么处理？

标准答案：可以把 token 存在 localStorage、sessionStorage 或 Cookie，也可以用 Pinia 持久化插件。应用启动时从本地恢复到 store。安全要求高时更推荐 HttpOnly Cookie，由后端控制。

面试加分点：localStorage 容易受 XSS 影响；HttpOnly Cookie 不能被 JS 读取，但要考虑 CSRF。

常见误区：认为 token 放 localStorage 就绝对安全。

代码示例：

```
const token = ref(localStorage.getItem('token') ?? '')

watch(token, value => {
  value ? localStorage.setItem('token', value) : localStorage.removeItem('token')
})
```

四、Axios 与前后端联调

16. Axios 为什么要二次封装？

问题：项目里封装 request 一般做什么？

标准答案：封装 Axios 是为了统一 baseURL、超时时间、token 注入、响应结构处理、错误提示、登录失效跳转。业务页面只调用接口方法，不关心通用请求细节。

面试加分点：响应拦截器不要随便吞掉错误，业务需要处理时要继续 `Promise.reject`。

常见误区：所有错误都统一弹“请求失败”，导致页面无法区分业务错误、401、500。

代码示例：

```
const service = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 15000
})

service.interceptors.request.use(config => {
  const token = useUserStore().token
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

service.interceptors.response.use(
  res => res.data,
  err => {
    ElMessage.error(err.response?.data?.message ?? '请求失败')
    return Promise.reject(err)
  }
)
```

17. 前后端联调常见问题怎么排查？

问题：接口联调失败时你怎么定位？

标准答案：先看浏览器 Network，确认 URL、Method、Headers、Payload、Status Code、Response。常见问题有跨域、路径不一致、方法不一致、参数格式不一致、枚举值类型不一致、时间格式不一致、token 过期、后端返回结构变化。

面试加分点：接口契约应通过 Swagger/OpenAPI、Apifox 或 YApi 管理，减少口头约定。

常见误区：只看控制台报错，不看 Network 里的真实请求和响应。

代码示例：

```
export interface PageResult<T> {
  total: number
  items: T[]
}

export function getUserPage(params: UserQuery) {
  return request.get<PageResult<UserDto>>('/api/users', { params })
}
```

18. 跨域是什么，怎么解决？

问题：CORS 报错的本质是什么？

标准答案：跨域是浏览器同源策略限制，协议、域名、端口任一不同都算跨域。解决方式通常是后端配置 CORS，开发环境也可以用 Vite proxy，生产环境建议同域部署或网关转发。

面试加分点：CORS 是浏览器限制，不代表服务器一定没收到请求；复杂请求会先发 OPTIONS 预检。

常见误区：只在前端加请求头，以为能解决 CORS。

代码示例：

```
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'https://dev.example.com',
        changeOrigin: true
      }
    }
  }
})
```

19. 401 登录过期怎么处理？

问题：多个请求同时返回 401 怎么办？

标准答案：在响应拦截器统一处理 401，清理 token 和用户信息，跳转登录页。要避免多个请求同时失败时重复弹窗、重复跳转。如果有 refresh token，需要做刷新队列；没有刷新机制就统一退出。

面试加分点：跳转登录时带上当前地址，重新登录后可以回到原页面。

常见误区：每个页面自己处理 401，逻辑重复且容易漏。

代码示例：

```
let redirecting = false

service.interceptors.response.use(undefined, error => {
  if (error.response?.status === 401 && !redirecting) {
    redirecting = true
    userStore.logout()
    router.replace(`/login?redirect=${router.currentRoute.value.fullPath}`)
  }
  return Promise.reject(error)
})
```

20. GET、POST、DELETE 参数怎么约定？

问题：查询、新增、批量删除分别怎么传参？

标准答案：查询通常用 GET query；新增编辑用 POST/PUT body JSON；单个删除可以用 DELETE path id；批量删除可以用 POST 到批量接口或 DELETE body，关键是前后端约定一致。空值、枚举、日期格式也要统一。

面试加分点：结合 C# 可以提到 [FromQuery]、[FromBody]、路由参数绑定的区别。

常见误区：前端传 JSON，后端按 query 接；前端传空字符串，后端期望 null。

代码示例：

```
request.get('/api/users', { params: query })
request.post('/api/users', form)
request.delete(`/api/users/${id}`)
request.post('/api/users/batch-delete', { ids })
```


五、Element Plus 后台系统

21. Element Plus 表单校验怎么做？

问题：新增、编辑弹窗表单如何校验？

标准答案：el-form 绑定 model 和 rules，el-form-item 配置 prop。提交时调用 formRef.validate()，通过后再调接口。关闭弹窗时清理表单和校验状态。

面试加分点：编辑回显后可能残留校验状态，可以在打开后 nextTick 调 clearValidate。

常见误区：el-form-item 忘写 prop，规则不生效。

代码示例：

```
<script setup lang="ts">
const formRef = ref<FormInstance>()
const form = reactive({ name: '', phone: '' })
const rules = {
  name: [{ required: true, message: '请输入姓名', trigger: 'blur' }]
}

async function submit() {
  await formRef.value?.validate()
  await saveUser(form)
}
</script>

<template>
  <el-form ref="formRef" :model="form" :rules="rules">
    <el-form-item label="姓名" prop="name">
      <el-input v-model="form.name" />
    </el-form-item>
  </el-form>
</template>
```

22. 新增和编辑共用弹窗怎么重置表单？

问题：如何避免编辑数据残留到新增？

标准答案：维护默认表单对象，打开新增时 Object.assign 重置字段，打开编辑时按字段回显。关闭弹窗时调用 resetFields 或 clearValidate。不要直接替换 reactive 对象。

面试加分点：编辑场景要注意 reset 调用顺序，否则可能把回显值当成初始值。

常见误区：直接写 form = {}，导致响应式引用丢失。

代码示例：

```
const defaultForm = { id: undefined, name: '', phone: '' }
const form = reactive({ ...defaultForm })

function resetForm() {
  Object.assign(form, defaultForm)
  formRef.value?.clearValidate()
}
```

23. 表格分页标准流程是什么？

问题：列表页查询、重置、分页怎么写？

标准答案：维护 `query`、`tableData`、`total`、`loading`。查询时页码重置为 1；分页变化后重新请求；重置时清空查询条件并回到第一页；接口请求期间显示 `loading`。

面试加分点：页码字段要和后端统一，比如 `pageIndex/pageSize` 或 `pageNum/pageSize`。

常见误区：筛选条件变化后不重置页码，导致明明有数据却查不到。

代码示例：

```
<script setup lang="ts">
const loading = ref(false)
const tableData = ref<UserDto[]>([])
const total = ref(0)
const query = reactive({ pageIndex: 1, pageSize: 10, keyword: '' })

async function loadData() {
  loading.value = true
  try {
    const res = await getUserPage(query)
    tableData.value = res.items
    total.value = res.total
  } finally {
    loading.value = false
  }
}
</script>

<template>
  <el-table v-loading="loading" :data="tableData" />
  <el-pagination
    v-model:current-page="query.pageIndex"
    v-model:page-size="query.pageSize"
    :total="total"
    @change="loadData"
  />
</template>
```

24. 下拉字典数据如何处理？

问题：状态、类型、性别这类字段怎么做？

标准答案：固定枚举可以前端常量维护；后台可配置字典应从接口获取。常用字典可以缓存到 `Pinia`。提交时传后端约定的值，展示时做 `label` 映射。

面试加分点：注意字典值类型统一，数字和字符串不要混用。

常见误区：后端返回 `1`，前端选项是 `'1'`，导致编辑回显失败。

代码示例：

```
const statusOptions = [
  { label: '启用', value: 1 },
```

```
{ label: '禁用', value: 0 }  
]
```

```
<el-select v-model="form.status">  
  <el-option  
    v-for="item in statusOptions"  
    :key="item.value"  
    :label="item.label"  
    :value="item.value"  
  />  
</el-select>
```

25. 表格多选和批量删除怎么做？

问题：批量删除如何获取选中行？

标准答案：使用 `type="selection"` 开启表格多选，通过 `selection-change` 保存选中行。批量操作前校验是否选择数据，二次确认后调用接口，成功后刷新列表。

面试加分点：普通表格多选只维护当前页，跨页选择需要额外设计。

常见误区：分页后以为上一页选择还保留；删除后当前页为空但没有回退页码。

代码示例：

```
<script setup lang="ts">  
const selectedRows = ref<UserDto[]>([])  
  
async function batchDelete() {  
  if (!selectedRows.value.length) return ElMessage.warning('请选择数据')  
  await ElMessageBox.confirm('确认删除选中数据吗?')  
  await deleteUser(selectedRows.value.map(x => x.id))  
  await loadData()  
}  
</script>  
  
<template>  
  <el-table :data="tableData" @selection-change="selectedRows = $event">  
    <el-table-column type="selection" width="55" />  
  </el-table>  
</template>
```

26. 弹窗组件如何封装？

问题：新增、编辑弹窗封装成子组件要注意什么？

标准答案：弹窗通过 `v-model` 控制显示，父组件传入 ID 或详情数据，子组件内部处理表单、校验和提交，保存成功后 `emit('success')` 通知父组件刷新列表。

面试加分点：可以用 `defineModel` 简化双向绑定；也可以用 `defineExpose` 暴露 `open` 方法。

常见误区：子组件直接修改父组件列表数据，组件耦合过重。

代码示例：

```
<script setup lang="ts">
const visible = defineModel<boolean>({ default: false })
const emit = defineEmits<{ success: [] }>()

async function submit() {
  await saveUser(form)
  visible.value = false
  emit('success')
}
</script>
```

六、工程化

27. Vite 环境变量怎么用？

问题：开发、测试、生产接口地址怎么区分？

标准答案：Vite 使用 `.env` 文件管理环境变量，暴露给前端的变量必须以 `VITE_` 开头。接口地址通常配置为 `VITE_API_BASE_URL`，代码中通过 `import.meta.env` 读取。

面试加分点：前端环境变量会被打包进静态资源，不能放数据库密码、后端密钥。

常见误区：把敏感密钥放前端 `.env`，以为用户看不到。

代码示例：

```
VITE_API_BASE_URL=/api
```

```
const baseUrl = import.meta.env.VITE_API_BASE_URL
```

28. TypeScript 在后台前端项目中的价值？

问题：为什么 Vue3 项目建议用 TypeScript？

标准答案：TypeScript 可以约束接口入参、返回值、组件 props、表单模型，减少字段拼错和接口变更带来的运行时错误。后台系统字段多、表单多、接口多，类型约束收益明显。

面试加分点：可以基于 OpenAPI 生成前端类型，减少手写 DTO 和后端不一致。

常见误区：大量使用 `any`，表面用了 TypeScript，实际没有类型保护。

代码示例：

```
export interface UserDto {
  id: number
  name: string
  status: 0 | 1
  createdAt: string
}
```

29. 前端项目目录怎么组织？

问题：Vue3 后台项目常见目录怎么分？

标准答案：api 放接口封装，views 放页面，components 放通用组件，router 放路由，stores 放 Pinia，utils 放工具函数，types 放类型，directives 放指令。核心是职责清晰、业务内聚。

面试加分点：中大型项目可以按业务模块组织，把页面、接口、类型放在同一模块下。

常见误区：所有接口堆一个大文件，所有工具函数都放 utils/index.ts。

代码示例：

```
src/  
  api/  
  views/  
  components/  
  router/  
  stores/  
  utils/  
  types/  
  directives/
```

30. 如何防止重复提交？

问题：用户连续点击保存按钮怎么办？

标准答案：前端用 loading 控制按钮禁用，在 finally 中恢复状态。关键业务后端还要做幂等控制，前端只能改善体验，不能作为最终防线。

面试加分点：订单、支付、审批这类操作必须以后端幂等为准。

常见误区：只靠前端禁用按钮保证不会重复提交，忽略抓包和并发请求。

代码示例：

```
const submitLoading = ref(false)  
  
async function submit() {  
  if (submitLoading.value) return  
  submitLoading.value = true  
  try {  
    await saveUser(form)  
  } finally {  
    submitLoading.value = false  
  }  
}
```

七、全栈衔接

31. 前端权限和后端权限如何分工？

问题：前端隐藏按钮后，后端还需要鉴权吗？

标准答案：必须需要。前端权限负责菜单、按钮、路由展示，提升用户体验；后端权限负责真正的安全控制，校验接口和数据访问。任何前端限制都可能被绕过。

面试加分点：数据权限也必须后端做，比如只能看本部门数据。

常见误区：认为按钮隐藏了，用户就无法操作接口。

代码示例：

```
const canDelete = hasPermission('user:delete')
```

```
[Authorize(Policy = "user:delete")]  
public Task DeleteUserAsync(long id) => service.DeleteAsync(id);
```

32. 统一响应结构怎么约定？

问题：接口返回一般怎么设计？

标准答案：常见响应包含 `code`、`message`、`data`、`traceId`。HTTP 状态码表达协议层状态，业务 `code` 表达业务结果。分页接口一般返回 `items` 和 `total`。

面试加分点：不要所有错误都返回 HTTP 200，401、403、500 应该使用合适状态码，方便网关和监控识别。

常见误区：只靠业务 `code` 表达所有错误，HTTP 状态码永远 200。

代码示例：

```
interface ApiResult<T> {  
    code: number  
    message: string  
    data: T  
    traceId?: string  
}
```

33. 时间格式和时区怎么处理？

问题：前后端时间差 8 小时怎么解决？

标准答案：前后端要统一时间标准。推荐后端存 UTC，接口返回 ISO 8601，前端按用户时区格式化展示。只传日期时要明确是日期还是时间点，避免偏移。

面试加分点：C# 后端可以使用 `DateTimeOffset` 表达明确时间点，减少时区歧义。

常见误区：前端手动加减 8 小时修问题，后续跨地区或夏令时会出错。

代码示例：

```
const text = new Intl.DateTimeFormat('zh-CN', {  
    dateStyle: 'medium',  
    timeStyle: 'short'  
}).format(new Date(row.createdAt))
```

34. 文件上传怎么实现？

问题：后台系统上传头像、附件、Excel 怎么做？

标准答案：前端使用 `el-upload` 或 `FormData` 以 `multipart/form-data` 提交，限制文件大小、类型、数量，并处理进度和错误提示。后端必须做文件校验、鉴权和存储，不能只信任前端校验。

面试加分点：大文件可做分片上传和断点续传；Excel 导入要返回错误行明细。

常见误区：只校验文件后缀，不校验文件内容和权限。

代码示例：

```
const data = new FormData()
data.append('file', file.raw)

await request.post('/api/files/upload', data, {
  headers: { 'Content-Type': 'multipart/form-data' }
})
```

35. Excel 导入导出怎么做？

问题：后台系统导出 Excel 如何处理？

标准答案：导出通常由后端生成文件流，前端设置 `responseType: 'blob'`，再创建下载链接。导入则上传文件，后端解析后返回成功数量、失败数量和错误明细。

面试加分点：大数据导出可以做异步任务，避免接口超时；文件名可从 `Content-Disposition` 读取。

常见误区：没有设置 `responseType: 'blob'`，导致下载文件损坏。

代码示例：

```
const res = await axios.get('/api/users/export', {
  responseType: 'blob',
  params: query
})

const url = URL.createObjectURL(new Blob([res.data]))
const a = document.createElement('a')
a.href = url
a.download = '用户列表.xlsx'
a.click()
URL.revokeObjectURL(url)
```

八、面试话术

36. 如何介绍自己做过的 RBAC 权限？

问题：请介绍你做过的基于路由的权限控制。

标准答案：我做过后台系统的 RBAC 权限控制。登录后后端返回用户信息、菜单和权限码；前端根据菜单生成动态路由和侧边栏；按钮通过权限码控制显示；路由守卫负责登录校验和动态路由初始化。我理解前端权限主要控制展示，接口和数据权限必须后端兜底。

面试加分点：补充刷新动态路由恢复、退出登录清理路由、401 统一处理。

常见误区：只说“隐藏按钮”，没有讲清菜单、路由、按钮、接口四层权限。

代码示例：

```
router.beforeEach(async (to) => {
  if (!userStore.token) return '/login'
  if (!permissionStore.initialized) {
    await permissionStore.initRoutes()
    return to.fullPath
  }
})
```

```
    return true
  })
```

37. 后端转全栈，如何表达前端能力？

问题：你 Vue3 不算特别深入，怎么证明能胜任全栈？

标准答案：我目前前端主要聚焦后台业务开发，熟悉 Vue3 Composition API、Vue Router、Pinia、Axios、Element Plus 表单表格和权限控制。后端经验让我更重视接口契约、权限边界、异常处理和数据一致性。前端深层源码不是我的主要优势，但常见业务场景和问题排查我能落地。

面试加分点：承认短板时，要同时说明掌握范围、项目经验和解决问题方法。

常见误区：只说“不深入”，没有说明自己能完成哪些具体前端工作。

代码示例：

回答结构：
掌握范围 -> 做过业务 -> 理解边界 -> 排查方法 -> 学习补齐方向

38. 一个后台列表页完整开发流程？

问题：如果让你做用户管理页面，你会怎么做？

标准答案：我会先确认接口契约和权限点，然后做查询表单、表格、分页、新增编辑弹窗、表单校验、删除确认、按钮权限、loading 和错误处理。最后联调空数据、异常、分页边界、权限不足和登录过期场景。

面试加分点：能主动提到边界场景，比只说“调接口渲染表格”更像实际开发。

常见误区：只关注页面显示，不关注权限、异常、loading、重复提交和分页边界。

代码示例：

查询条件 -> 请求接口 -> 表格渲染 -> 分页 -> 新增编辑 -> 校验 -> 权限 -> 异常处理

九、补充章节（claude-mimo 文档独特内容）

与前文重复的题（响应式原理、ref/reactive、computed/watch、nextTick、组件通信、路由守卫/RBAC、hash/history、Pinia 持久化、Axios 封装、跨域、401、Element Plus 表单/分页/弹窗、Vite 环境变量、文件上传、前后端分工）已并入上文，以下为原文档未覆盖的补充题。

39. Composition API 和 Options API 的区别？

Options API 按 data/methods/computed 等选项组织代码，逻辑分散在不同选项里；Composition API 用 setup() 函数按功能逻辑组织代码，相关逻辑写在一起。Vue 3 两种都支持，推荐 Composition API。

加分点：

实际项目中我用 <script setup> 语法糖，它比普通 setup 更简洁，编译时自动暴露变量，不用 return。Options API 在简单组件里也能用，不用非此即彼。

常见误区：

- 以为 Composition API 是 Options API 的"替代品"——两者共存

- 以为 `<script setup>` 是一个新 API——它只是语法糖，底层还是 Composition API
- 以为 `setup` 里不能用生命周期——可以用 `onMounted`、`onUnmounted` 等

40. Vue 3 的生命周期有哪些？

`setup` 本身就是最早的阶段。Composition API 中用 `onBeforeMount`、`onMounted`、`onBeforeUpdate`、`onUpdated`、`onBeforeUnmount`、`onUnmounted`。`onMounted` 最常用，适合发请求、操作 DOM。

加分点：

实际开发中我最常用 `onMounted` 发起初始化请求，`onUnmounted` 清理定时器和事件监听。
`<script setup>` 里直接调用就行，不需要像 Options API 那样写在特定选项里。

常见误区：

- 以为 `setup` 里写代码就等于 `created`——`setup` 等价于 `beforeCreate` + `created`，但概念不同
- 忘记在 `onUnmounted` 中清理副作用——容易造成内存泄漏
- 在 `onUpdated` 里修改响应式状态——容易死循环

41. Pinia 和 Vuex 的区别？为什么推荐 Pinia？

Pinia 是 Vue 官方推荐的状态管理库（Vue 3 默认）。相比 Vuex：去掉了 mutations，只有 state + getters + actions；TypeScript 类型推导更好；支持多 store 拆分；体积更小；支持组合式 API 风格。

加分点：

Pinia 最大的好处是简单。Vuex 那套 `commit mutation` → `dispatch action` 的流程太绕了，Pinia 直接在 actions 里改 state 就行。devtools 支持也好，可以追踪每次状态变化。

42. Vite 和 Webpack 的区别？为什么现在都用 Vite？

Vite 开发环境用浏览器原生 ESM，不需要打包，冷启动极快；Webpack 需要先打包再启动。Vite 生产环境用 Rollup 打包。Vite 配置简单，Vue 3 + Vite 是目前主流方案。

加分点：

Vite 的核心优势是开发体验。HMR（热更新）几乎是即时的，因为只更新改动的模块。大型项目 Webpack 冷启动可能要几十秒，Vite 几秒就起来了。不过 Webpack 的生态和插件还是更成熟。

常见误区：

- 以为 Vite 生产环境也不打包——生产环境还是用 Rollup 打包
- 以为 Vite 完全替代了 Webpack——大型老项目迁移成本高
- 以为 Vite 不需要配置——alias、proxy、环境变量还是需要配置

43. 前端项目怎么优化打包体积？

- 路由懒加载：`() => import('./views/User.vue')`
- 组件按需引入：Element Plus 用 `unplugin-vue-components`
- 图片压缩、CDN 加速、gzip/brotli 压缩
- Tree Shaking（Vite 默认开启）

加分点：

路由懒加载最容易见效，配合 Vite 的代码分割，首屏只加载当前路由需要的代码。Element Plus 按需引入可以把体积从 800KB 降到 200KB 左右。用 `rollup-plugin-visualizer` 分析打包产物。

```
const routes = [
  { path: '/user', component: () => import('@views/User.vue') },
  { path: '/order', component: () => import('@views/Order.vue') }
]
```

44. 前端如何处理 Token 过期（refresh token 队列）？

请求拦截器带 token，响应拦截器判断 401。短期 token 过期用 refresh token 换新 token，refresh token 也过期则跳登录页。**关键是并发请求的处理：多个请求同时 401，只需要刷新一次 token**，用 `isRefreshing` 标记 + pending 队列，刷新完成后重发队列里的请求。

```
let isRefreshing = false
let pendingQueue = []

service.interceptors.response.use(
  response => response.data,
  async error => {
    const { config, response } = error

    if (response?.status === 401) {
      if (isRefreshing) {
        // 正在刷新，加入队列等待
        return new Promise(resolve => {
          pendingQueue.push(() => resolve(service(config)))
        })
      }

      isRefreshing = true
      try {
        const refreshToken = localStorage.getItem('refreshToken')
        const res = await axios.post('/api/auth/refresh', { refreshToken })
        localStorage.setItem('token', res.data.token)
        localStorage.setItem('refreshToken', res.data.refreshToken)
        // 重发队列中的请求
        pendingQueue.forEach(cb => cb())
        pendingQueue = []
        return service(config) // 重发当前请求
      } catch {
        // refresh 也失败，跳登录
        localStorage.clear()
        window.location.href = '/login'
      } finally {
        isRefreshing = false
      }
    }

    return Promise.reject(error)
  }
)
```

45. 大文件上传怎么实现（切片 + hash + 并发）？

前端切片（`File.slice`）→ 计算文件 hash → 并发上传切片 → 通知后端合并。支持断点续传（记录已上传切片）、秒传（hash 比对）、暂停/恢复。

加分点：

用 Web Worker 计算文件 hash 避免卡主线程。上传并发数控制在 3-6 个，太多浏览器会排队。每个切片独立请求，失败可以单独重试，不用重传整个文件。

```
async function uploadFile(file) {
  const CHUNK_SIZE = 5 * 1024 * 1024 // 5MB
  const chunks = []
  for (let i = 0; i < file.size; i += CHUNK_SIZE) {
    chunks.push(file.slice(i, i + CHUNK_SIZE))
  }

  const hash = await calcHash(file)
  const { uploaded } = await api.checkFile(hash) // 秒传/断点判断
  if (uploaded.length === chunks.length) {
    return // 秒传成功
  }

  const maxConcurrent = 4
  let index = 0

  async function next() {
    if (index >= chunks.length) return
    const i = index++
    if (uploaded.includes(i)) { await next(); return } // 跳过已上传
    const formData = new FormData()
    formData.append('chunk', chunks[i])
    formData.append('hash', hash)
    formData.append('index', i)
    await api.uploadChunk(formData)
    await next()
  }

  await Promise.all(Array.from({ length: maxConcurrent }, () => next()))
  await api.mergeFile({ hash, filename: file.name, total: chunks.length })
}
```

46. 前端怎么做国际化 (i18n) ?

用 vue-i18n 插件，定义语言包 (JSON)，模板中用 `$t('key')`，JS 中用 `useI18n()`。切换语言时修改 locale。

加分点：

Element Plus 也有自己的国际化，需要配合 `el-config-provider` 的 `locale` 一起切换。语言包按模块拆分，不要一个大文件。用户语言偏好存 localStorage。

常见误区：

- 硬编码中文在模板里
- 只翻译页面文字，忘了翻译 Element Plus 组件
- 语言包太大——按需加载，不用的语言包不要打包

47. 前端怎么做数据可视化?

常用方案：ECharts (功能最全)、AntV (阿里系)、Chart.js (轻量)。后台系统一般用 ECharts，配合 `vue-echarts` 封装。大屏用 DataV。

加分点：

ECharts 的坑主要是 `resize`。窗口大小变化时图表不会自适应，需要监听 `resize` 事件手动调 `chart.resize()`。组件销毁时要 `chart.dispose()` 避免内存泄漏。

```
onMounted(() => {
  chartInstance = echarts.init(chartRef.value)
  chartInstance.setOption({ /* ... */ })
  window.addEventListener('resize', handleResize)
})

onUnmounted(() => {
  window.removeEventListener('resize', handleResize)
  chartInstance?.dispose() // 必须销毁
})
```

48. 微前端了解吗？什么时候需要用？

微前端是把多个独立的前端应用组合成一个应用。主流方案：qiankun（基于 single-spa）、Module Federation（Webpack 5）、iframe。适用于多团队维护、技术栈不统一、需要独立部署的场景。

加分点：

qiankun 是国内最常用的方案，阿里出品。核心概念是主应用（基座）+ 子应用。主应用负责路由分发和公共依赖，子应用独立开发部署。坑主要是样式隔离和 JS 沙箱的兼容性。

常见误区：

- 小项目也上微前端——过度设计
- 以为微前端解决所有问题——其实引入更多问题（通信、样式冲突、性能）
- 只了解概念不了解原理——答不出 JS 沙箱和样式隔离的实现

49. 后台管理系统登录流程怎么设计？

登录页提交账密 → 后端返回 token + refresh token → 前端存储 token → 请求拦截器带 token → 响应拦截器处理 401 → refresh token 续期或跳登录 → 退出登录清除 token 和用户状态。

加分点：

token 存储我倾向于 httpOnly cookie（防 XSS）而不是 localStorage。如果后端不支持 cookie，localStorage 也可以，但要注意 XSS 防护。登录成功后要获取用户信息和权限，再根据权限动态添加路由。

常见误区：

- token 存在 Pinia 里不持久化——刷新就丢了
- 退出登录不清路由——要用 `router.removeRoute` 清理动态路由
- 登录页不做路由守卫——已登录用户可以直接访问登录页

50. 如何封装一个通用的 Table + Search + Pagination 组件？

把搜索条件、表格配置、分页参数封装成一个组件，通过 props 传入列配置和搜索字段定义，内部管理 loading、数据请求、分页逻辑，对外暴露 refresh 方法。

加分点：

后台管理系统里 80% 的页面都是「搜索 + 表格 + 分页」，封装后一个页面只需要配置列定义和接口地址。可以用 `v-bind="$attrs"` 透传 el-table 属性，用插槽支持自定义列。

常见误区：

- 封装得太死——不同页面差异大时组件难以维护
- 不支持插槽自定义——实际项目中很多列需要自定义渲染
- 搜索和分页不联动——搜索后要重置到第一页

51. 面试话术总结

场景	话术模板
----	------

回答问题	"这个问题我的理解是...在实际项目中我是这样做的..."
------	-------------------------------

不确定时	"这块我了解基本原理，实际用的不多，我的理解是..."
------	-----------------------------

展示经验	"之前项目里遇到过一个场景..."
------	-------------------

承认不足	"这块我还在深入学习中，目前的了解是..."
------	------------------------

反问面试官	"想了解一下贵司前端的技术选型和项目规模"
-------	-----------------------